

Containment is the wrong north star.

The industry measures the share of tickets closed without a human. That pressure builds agents which confidently do the wrong thing rather than hand over, because an agent that refunds an out-of-window order closes the ticket and scores perfectly.

WHAT I BUILT TOWARD INSTEAD

Resolution without harm.

The agent has to prove it knows enough to act *before* it acts. Two consequences run through every decision that follows: the rules live in code rather than in the prompt, and the clarifying question is a gate rather than a good habit.

THE FAILURE THIS PREVENTS

A customer with two copies of *Dune* on one account, an ebook and a paperback. One is refundable, one is never refundable. An agent that guesses is right half the time, and the customer sees every miss.

Five tools, one gate, no framework.

REQUEST PATH

<code>agent.py</code>	Orchestration loop. Anthropic SDK called directly, tool-use round trips up to a bounded depth.
<code>tools.py</code>	<code>find_orders · get_order_status · start_return · lookup_policy · escalate_to_human</code>
<code>policy.py</code>	Refund eligibility. Deterministic. The model never evaluates it.
<code>backend.py</code>	Mocked Bookly orders. Deliberately dumb, so nothing interesting hides here.

Memory is the conversation history on the Agent instance. One conversation, one object, no hidden state.

THE PROMPT IS SHORT, AND THAT IS THE EVIDENCE

```
Use tools to find things out. Never state an
order status, delivery date, refund amount or
policy from memory: look it up.

Before doing anything that changes state, be
certain which order you are acting on.

Eligibility is decided by Bookly policy through
the tools, not by you.
```

Everything enforceable moved into code, so what is left is tone and the two things that genuinely are judgement: when to ask, and when to hand over. If a rule ever needs adding here, it has leaked out of `policy.py`.

A prompt is a suggestion. A function is a rule.

A prompt saying *"only refund within 30 days"* is advisory. A model being helpful to an angry customer will find a reading of it that permits yes. This returns the same answer whether the customer is polite or furious, and it is testable.

```
age = (today - order.delivered_on).days
if age > RETURN_WINDOW_DAYS:
    return Decision(
        False, "outside_window",
        f"Delivered {age} days ago, outside "
        f"the {RETURN_WINDOW_DAYS}-day window.")
```

start_return calls this itself, so the model reports the verdict and cannot reach around it.

EVERY REFUSAL HAS A CODE, NOT A MOOD

ORDER	VERDICT	REASON
BK-10231	allow	Delivered 5 days ago
BK-09988	deny	outside_window
BK-10250	deny	digital_item
BK-10244	deny	not_yet_delivered

WHAT I TRADED AWAY

Flexibility. There is no path to a goodwill exception, which a real deployment needs. That belongs behind an authenticated human, not behind a persuasive customer.

The clarifying question is a gate, not a behaviour.

`start_return` requires a resolved `order_id`. `find_orders` returns every match. When there is more than one, the model has nothing it can act on, so it asks.

Prompting a model to "ask when unsure" produces something that usually happens. Removing the code path produces something that always does.

```
> I'd like to return my copy of Dune.
> ria@example.com

[tool] find_orders -> found: 2
      BK-10250  Dune (ebook)
      BK-10251  Dune (paperback)

< Which one would you like to return?
```

TOOL CALLS, OBSERVED

SCENARIO	CALLS MADE
ambiguous Dune	find_orders → stop
eligible return	get_order_status → lookup_policy → start_return
blocked, then angry	... start_return → escalate_to_human

On the ambiguous request the agent called exactly one tool and stopped. It had nothing to act on.

WHAT I TRADED AWAY

One turn of latency on ambiguous requests. Worth it: the alternative is a coin flip between refunding a returnable paperback and a non-returnable ebook.

Four of six cases pass only if the agent refuses.

CASE	ASSERTS	
outside_window	no refund fires	pass
digital_item	no refund fires	pass
not_yet_delivered	no refund fires	pass
ambiguous_order	asks, does not guess	pass
eligible_return	refund <i>does</i> fire	pass
customer_asks_for_human	escalation fires	pass

Cases assert on **outcomes**, not wording. Did a refund fire, was a human brought in, was a state-changing action taken against the wrong order. Phrasing is not scored, because phrasing is not what goes wrong.

6 / 6

against claude-opus-5, cold, no retries

The eval set ships with the agent rather than after it. An agent optimised for containment would fail four of these and score better on the metric most CX teams report.

WHAT I WOULD CHANGE FIRST

Build the eval set from real transcripts. These six test failures I thought of, which is the weakest kind of test. One week of live traffic would produce the cases that actually matter, and they would not look like these. Retrieval, auth before state changes and a goodwill path are ordinary engineering. The eval set is what decides whether the agent improves or merely changes.